RecoveryTrac Mobile Automation

End-to-end UI tests for **RecoveryTrac Flex** (collection / disposal monitor) and the **RecoveryTrac Driver App**, written in C# with NUnit, Selenium and Appium (UiAutomator2). The same test runs in two modes:

Mode	Flex runs in	Driver App runs in
Web	Chrome on `http://localhost:8100`	Edge on `http://localhost:8200`
Mobile	Node 1: `emulator-5554` (Android 14) Node 2: `emulator-5558` (Android 15) Node 3: `emulator-5562` (Android 17)	Node 1: `emulator-5556` (Android 12) Node 2: `emulator-5560` (Android 13) Node 3: `emulator-5564` (Android 16)

Both modes run against the **local Aspire backend**.

What the end-to-end test covers

`Tests/LoadTicketCollectionTests.cs` → `Test_LoadDisposalCollectionFlow_And_VerifySubmittedSynced_E2E`, as three test cases that run **in parallel**, one per node pair:

Test case	Flex	Driver App	Flex account
`..._E2E_Node1_A14_A12`	`Grid_Node1_Flex` (Android 14)	`Grid_Node1_Driver` (Android 12)	`TestSettings:LoginEmail`
`..._E2E_Node2_A15_A13`	`Grid_Node2_Flex` (Android 15)	`Grid_Node2_Driver` (Android 13)	`TestSettings:Node2:LoginEmail`
`..._E2E_Node3_A17_A16`	`Grid_Node3_Flex` (Android 17)	`Grid_Node3_Driver` (Android 16)	`TestSettings:Node3:LoginEmail`

Each node logs in with its **own Flex account**. With a shared account, each node would see the others' submitted tickets, and the "exactly one new ticket" check would fail. That's why Node 2 and Node 3 stop at setup with a clear message if their account isn't set. Only Node 1 uses the default account.

Each test case runs these steps:

1. **Flex: log in**, accept the Privacy Policy and Terms of Service, and check that a project is assigned.
2. **Flex: Load - Collection ticket.**
    - Truck: picks the first truck; the form fills in the truck's details.
    - Trailer: answers **No**.
    - Collection Start: adds the debris pile photo.
    - Collection Information: debris type **Vegetative/Woody**.
    - Location Information: captures the GPS location (mocked to Polk City Rd, FL) and waits for the street name.
    - Submits, generates the QR code and saves it, then waits for the ticket to reach **Synced**.
3. **Driver App:** GET STARTED → SKIP, then scans the collection QR. The scanned ticket becomes the **Current Load**.
4. **Flex: Load - Disposal ticket.**
    - Import Collection Data: scans the Driver's QR, then picks a disposal site.
    - Truck Container Load Call: Load Call **50%**, plus a photo.
    - Submits, waits for **Synced**, then generates the disposal QR and saves it.

5. **Driver App:** Scan Receipt Code with the disposal QR. The receipt must match the tickets: receipt number, truck, debris type, capacity and load call.

Generated QR codes are saved under `bin/Debug/net10.0/QrCodes/` (named with the node, e.g. `CollectionLoadTicket_Node2_<timestamp>.png`) and attached to the test result.

Steps 1–2 come from `Flows/CollectionLoadTicketFlow.cs` and step 4 from `Flows/LoadDisposalFlow.cs`. The QR baselines below use the same flows.

QR baselines (QR-B / QR-D)

These tests record how the QR handoff behaves **today**, so the BLE tests have an expected result to compare with (for example "the Driver App behaves as after a QR scan"). Each one saves what it observed as JSON under `bin/Debug/net10.0/Baselines/` and only fails on what must hold for QR too. Something worth knowing that isn't a failure (e.g. a damaged code that is accepted) appears as a *Finding* in the report. The scenarios are described in Part A of `docs/testing/test-scenarios-ble-ticket-handoff.md`.

Fixture	Scenarios	Covers	Category
`Tests/QrBaselineTests.cs`	QR-B01…B11	Collection leg: collection QR → Driver App load (payload, load after restart, second ticket, re-scan, non-ticket and damaged codes, airplane mode, abandon prompt)	`QR-Baseline`
`Tests/QrDisposalBaselineTests.cs`	QR-D01…D10	Disposal leg, ahead of BLE load disposal: Driver load QR → Flex *Load - Disposal* import (BLE *Receive Data*), and disposal QR → Driver App receipt (BLE *Send Receipt*)	`QR-Baseline`, `QR-Baseline-Disposal`

The disposal scenarios:

ID	Records	Node
QR-D01 / D02 / D03	The Driver's load QR payload and what the disposal form imports; the disposal QR payload; the disposal QR reopened from its card carries the same payload	Node 2
QR-D04 / D05	The receipt after the scan; the receipt still listed after a restart, and whether the receipt ends the load	Node 3
QR-D06	The same disposal QR scanned twice	Node 2
QR-D07	The disposal form with a code that isn't a load, and with a load payload cut in half or with one character changed	Node 3
QR-D08	*Scan Receipt Code* with a code that isn't a disposal ticket, and with damaged disposal payloads	Node 2
QR-D09	A driver holding load A scans the disposal QR of load B (BLE: a receipt sent to the wrong Driver App)	Node 3
QR-D10	Disposal and receipt with both devices in airplane mode	Node 2

Each fixture runs on Node 2 and Node 3 in parallel, logged in with that node's own Flex account; tests on the same node pair run one after the other. QR-B06, B07, B09, D07, D08 and D10 need Mobile mode (they stage arbitrary QR text or use airplane mode) and are ignored in Web mode. Helpers shared by both fixtures (submitting a ticket, damaged payload variants) are in `Tests/QrBaselineTestBase.cs`.

BLE ticket handoff (TC-12 / TC-13)

`Tests/BleTicketHandoffTests.cs` → `BLE_TC12_TC13_CollectionTicket_SentViaBluetooth_IsLoadedOnDriver` (category `BLE`). This is Part B of `docs/testing/test-scenarios-ble-ticket-handoff.md`. The collection ticket goes from Flex to the Driver App **over Bluetooth instead of a QR scan**. The test runs only in Mobile mode; in Web mode it is skipped.

Fixture	Node pair	Flex account
`BleTicketHandoffTests("Node2","testuser2")`	`Grid_Node2_Flex` (Android 15) + `Grid_Node2_Driver` (Android 13)	the configured account whose email is `…testuser2@…`
`BleTicketHandoffTests("Node3","testuser4")`	`Grid_Node3_Flex` (Android 17) + `Grid_Node3_Driver` (Android 16)	the configured account whose email is `…testuser4@…`

The two fixtures run **one after the other** (`[NonParallelizable]`): each test stops every grid Driver App to tell its own apart (see *Shared Bluetooth* below). Each fixture uses its own Flex account. Neither sees the other's tickets, nor the tickets of Node 1's QR E2E, which uses `testuser1`. The account name is matched against the `LoginEmail` values in user-secrets (`TestSettings:LoginEmail` or any `TestSettings:<name>:LoginEmail`). Its section name doesn't matter, but the account must be set there.

What each run covers:

Check	Covered by
TC-12: happy path	Flex sends a submitted *Load - Collection* ticket via Bluetooth. The Driver App shows it as its **Current Load**, with a "*Load <number> received.*" toast
TC-12: Bluetooth access	The Flex permission is reset before the run. Android must ask "*Allow Flex to find nearby devices?*" the first time Flex uses Bluetooth (straight away, or after **Continue** on Flex's *Bluetooth Access Required* alert). The test taps **Allow** and checks the permission is granted
TC-12: Driver App card	Flex lists this node's Driver App as an identity card (the truck's details, or *Driver App + Identity Code*). The status must read **Sent to <Identity Code>**, then **Done** closes the menu
TC-12: timing (check 2)	Time from picking the truck to the load showing on the Driver App. Over **5 s** gives a **warning**, not a failure: emulator timings are indicative only, and sign-off happens on the demo phones
TC-13: payload parity	The Driver App's load QR decodes to exactly the same payload as the same ticket's QR in Flex. Both SHA-256 hashes are recorded

Steps:

1. **Flex:** resets and relaunches Flex (Nearby devices permission revoked), logs in, then fills in and submits a *Load - Collection* ticket. Steps 1–17 use the same flow as the QR tests (`Flows/CollectionLoadTicketFlow.cs`).
2. **Flex:** declines the QR prompt and finds the ticket under **Submitted**, then waits for it to reach Synced.
3. **Flex:** stops the Driver App on every grid emulator (granting it Nearby devices), taps the submitted card's **Bluetooth** icon, taps **Allow** on Android's question (and **Continue** on *Bluetooth Access Required* if Flex shows it). The Nearby menu (*Send via Bluetooth*) searches; what it lists now is other devices.
4. **Driver App:** GET STARTED → SKIP and cancels any load left over from an earlier run. There is no *Receive via Bluetooth* button: the app is discoverable while it is open.
5. **Flex: Try Again**, waits for the new card (this node's Driver App) to finish *Loading truck details…*, taps it, and waits for **Sent to …**.
6. **Driver App:** waits for the Current Load and reads the toast and the load's QR. Back in Flex, the test opens the same ticket's QR as the reference payload.

Each run saves its measurements to `bin/Debug/net10.0/Baselines/BLE-TC12_<node>_<timestamp>.json`: ticket, the Driver App card, send result, `sentSeconds` / `loadedSeconds`, toast, payload length and both payload hashes.

Not covered yet: the disposal side of the BLE handoff (still in development), so the test ends once the Driver App holds the load. Its QR expected results are already recorded by the QR-D baselines (see QR baselines). The ×10 iteration run and the "≥ 9 of 10 under 5 s" sign-off from the test plan are also out of scope. Emulator timings are indicative only.

Shared Bluetooth: all emulators share one simulated Bluetooth, so each Flex would also list the other node's Driver App, and the Driver App doesn't show the Identity Code Flex lists it by. So the test finds its own Driver App by difference: it stops every grid Driver App, notes what Flex lists, starts its own and takes the card that appears.

Feature #10082 R1-C1: Driver App Nearby discoverability (FEAT-NDT-1)

`Tests/NearbyDiscoverabilityTests.cs` (category `BLE-R1-C1`) checks the R1 requirements from the #10082 requirements plan. These tests check the **requirement**, and each failure names its requirement ID, so a run doubles as the gap report. With the current debug builds (the Driver App discoverable in the foreground, no *Receive via Bluetooth* button) all five pass; the last column shows the latest run (2026-11-06, Node 2).

Test	Requirement	Driver App state	Current builds
`NDT1_P01_N02_DriverOnHome_IsDiscoverable_WithoutReceiveMode`	10082-NDT-1-P01, N02	Open on Home, *Receive via Bluetooth* never tapped; no alert, pairing request or permission dialog on its screen	✅
`NDT1_P02_DriverOnEveryPage_StaysDiscoverable`	10082-NDT-1-P02	Home, then Receipts, then back on Home: listed on each	✅
`NDT1_N01_DriverBeforeHome_IsDiscoverable`	10082-NDT-1-N01	Open on Welcome, then on the Tutorial, before Home (the Tutorial preference is reset first so it shows on every emulator; a warning if it still does not)	✅
`NDT1_N01_DriverOnWelcome_ReceivesLoad_AndNotifiesDriver`	10082-NDT-1-N01 (+ Story 4.4 receipt toast)	Driver App left on Welcome receives a new ticket: Flex shows *Data sent to…*, the Driver App shows *Load <n> received.* while still on Welcome, and after GET STARTED Home shows the load with the same payload as its QR	—
`NDT1_P03_DriverBackFromBackground_IsDiscoverableAgain`	10082-NDT-1-P03	Listed, then sent to the background, then brought back with nothing tapped. Whether it stays listed while in the background is only recorded (today: not listed)	✅
`NDT1_P03_DriverScreenTimedOut_NotDiscoverable_ThenDiscoverableOnUnlock`	10082-NDT-1-P03	Listed on Home with no load, then left until the screen times out (set to 15 s for the test). Under Appium the Driver window keeps the screen on, so after 40 s it is locked with the power key instead (same state: screen off, app paused). Locked, it must not be listed. After the phone is unlocked it is listed again with nothing tapped in the app	✅
`NDT1_Control_DriverStarted_IsFoundByDifference`	Control	Started the way the transfer tests start it (the POC's receive mode only in a build that still has the button)	✅

Pass means Flex's Nearby menu (*Send via Bluetooth*) lists the Driver App's card within 25 s of opening (it shows up almost at once on the emulators).

How a test finds its own Driver App. Without receive mode the Driver App shows no code to match:

1. The test stops the Driver App on **every** grid Driver emulator, because all emulators share one simulated Bluetooth. It also grants the Driver App its Nearby devices permissions there, because NDT-1 only applies while that access is available.
2. It opens the Nearby menu on any submitted *Load - Collection* ticket, submitting one only if the account has none, notes what that search lists, and closes it.
3. It starts this node's Driver App and opens the Nearby menu again for each check (a new discovery attempt each time, so a card can't be left over from an earlier check). The new card is this node's Driver App.

The **control** test runs the same search with the Driver App started as the transfer tests start it. While it passes, a failing NDT-1 test means the Driver App wasn't advertising in that state; the search itself works.

The fixture runs on Node 2 (`testuser2`) and is `NonParallelizable` . Don't run it next to the BLE handoff tests: it stops their Driver Apps.

Not automated in R1-C1:

- R1-INFRA-SPIKE is a go/no-go decision. QR-B10 (`DeviceBluetoothReadinessTests`) and TC-12/13 are its evidence.
- R1-INFRA-FLAG is blocked: no app has a #10082 flag yet. Once one exists, check that a flag-off build hides every #10082 entry point in Flex, the Driver App and Form Builder, and that the QR flows still match `QrBaselineTests` .

Details are in Part C of `docs/testing/test-scenarios-ble-ticket-handoff.md` .

Feature #10082 R1-C2: Transfer core (FEAT-NDT-2, 3, 5, 6, 10)

Category `BLE-R1-C2` covers Flex's side of the transfer: the discovery menu, the Driver App cards and the Bluetooth access gate. As in R1-C1, these tests check the **requirement**, and each failure names its requirement ID.

Pending: **`BLE-R1-C2-Pending`.** The two NDT-10 tests (progress view and Cancel) are in their own category until Flex ships **Story 4.8**. Until then the send view keeps the earlier *Send via Bluetooth* title, *Connecting to <code>…* / *Sending…* with a progress bar, and only a disabled *Close*, so they fail on purpose (each failure names its NDT-10 requirement). `Category=BLE-R1-C2` doesn't include them. Run them on purpose with `-Filter "Category=BLE-R1-C2-Pending"` ; once 4.8 is in the APK and they pass, move `NearbyTransferProgressTests` back to `BLE-R1-C2` .

Defects: **`BLE-R1-C2-Defects`.** Tests for Driver App defects found during BLE testing. They fail until the defect is fixed, so `Category=BLE-R1-C2` doesn't include them. Run them with `-Filter "Category=BLE-R1-C2-Defects" -Nodes 2` .

Test	Requirements	What it does
`NearbyDiscoveryMenuTests.NDT2_NoDriverApp_NoResultsAfter10s_TryAgain_DismissStopsDiscovery`	NDT-2-P01, P02, E01, E02, E03, E05, P16, N04	No Driver App nearby: searching starts at once, the no-results state appears after 10 s, Try Again starts a new 10-s attempt, and dismissing mid-search stops discovery and leaves the ticket as it was
`NearbyDiscoveryMenuTests.NDT2_NDT3_DriverAppFound_ListedWhileDiscoveryContinues`	NDT-2-P03, E06, P04, N02, N01; NDT-3-P01, P03, P21, P12	The Driver App's card shows under the running search and stays past 10 s while the search continues. It shows *Driver App*, its *Identity Code* and a distance, and disappears by itself once the Driver App closes

Test	Requirements	What it does
`NearbyTransferProgressTests.NDT10_SelectDriverApp_StopsDiscovery_ShowsSendDataProgress` (**pending**)	NDT-2-P14, N04; NDT-10-P01, P02, P03, P04, P06, P07, N01	Picking the Driver App stops discovery. The same menu then shows *Send Data*, *Connecting…* and *Sending data…* with a spinner and Cancel, and no percentage
`NearbyTransferProgressTests.NDT10_CancelDuringTransfer_ReturnsToUnchangedTicket` (**pending**)	NDT-10-P08, N02, P09, P10, N03, E01	A tap outside and Escape don't dismiss the menu. Cancel returns to the unchanged ticket with no load on the Driver App, or success wins if the Driver App committed first
`NearbyReceiveDuringAbandonTests.NearbyLoad_ReceivedWhileAbandonPromptOpen_IsKeptAfterConfirm` (**defect**)	— (pending PO confirmation)	The Driver App holds load A, and the driver taps Cancel Load. While Confirm counts down, Flex sends load B, and the Driver App shows *Load B received*. Tapping Confirm must leave B as the active load, with the same payload as its QR. Today it removes B, and Home shows no load
`QrBaselineNode3Tests.QR_B11_SecondTicketDuringAbandonPrompt_BehaviorIsRecorded` (QR counterpart)	— (QR-B11 baseline)	The same case over QR. With load A's abandon prompt open, a tap on *Scan Receipt Code* lands on the prompt's backdrop and closes the prompt, so no scan starts and load A stays. Confirm then removes A. Ticket B, scanned with *Scan QR Code*, becomes the load with its QR payload. With QR, no load can arrive behind the prompt
`NearbyRuntimeAccessTests.NDT6_BluetoothOff_ShowsAccessFlow_ThenSearchesOnceOn`	NDT-6-P01, P02, P03, P08, N01, P09	Bluetooth off: the Bluetooth Access flow appears instead of a search; once Bluetooth is on, the search starts without tapping the Nearby action again
`NearbyRuntimeAccessTests.NDT6_BluetoothOff_CancelAccessFlow_ReturnsWithoutSearching`	NDT-6-E02	Cancel in the access flow returns to the ticket list and no search starts, even after Bluetooth comes back
`NearbyRuntimeAccessTests.NDT6_NearbyAccessDenied_DoesNotSearch`	NDT-6-P02, E03, N01	The user refuses Nearby devices access: no search starts
`NearbySupportedClientsTests.NDT5_FlexWithoutNearby_KeepsQrAction_ShowsNoNearbyAction`	NDT-5-P05, P06, N01	`RunMode=Web` **only** (ignored on devices): Flex Web shows no Nearby action, not even a disabled one, and its QR action still works

How they work:

- **Texts, not tags.** The R1 menu isn't built yet, so `Pages/NearbyDriverMenuPage.cs` reads the top Flex overlay by the texts the requirements define. A recorder inside the WebView logs every change, so short states such as *Connecting…* are caught and timed.
- **Scan state from Android.** "Discovery stopped" is also checked from Android's own record of Flex's Bluetooth scans (`BluetoothHelper.IsScanning`).
- **Same Driver App setup as R1-C1.** The tests stop the grid Driver Apps and find this node's Driver App by difference. The Driver App uses the POC's *Receive via Bluetooth* while that button exists, so an R1-C1 gap doesn't hide the R1-C2 results.
- **The device is put back.** The NDT-6 tests turn the Flex emulator's Bluetooth off or revoke its Nearby devices access, and restore both in `finally` .

All of them except the Web one run on Node 2 (`testuser2`) and are `NonParallelizable` . Don't run them next to the BLE handoff tests: they stop those tests' Driver Apps.

Not automated in R1-C2: the alternative-action requirements, which wait on an open question about which alternative R1's entry point offers; distance sorting, press deferral and removal animations; and Driver Session identity cards (`driverSession` is off in every build). Part D of `docs/testing/test-scenarios-ble-ticket-handoff.md` lists each requirement with its reason.

Project layout

Path	Contents
`Tests/`	Test fixtures (`BaseTest` closes the apps and sessions after each test)
`Pages/`	Flex page objects (`LoginPage`, `PrivacyPolicyPage`, `HomePage`, `FormsPage`, `CollectionLoadTicketFormPage`, `LoadDisposalFormPage`, ...)
`Pages/NearbyDriverMenuPage.cs`	#10082 R1 Nearby menu (discovery, progress, Bluetooth Access flow), read by the required texts and recorded from inside the WebView
`Tests/NearbyTestBase.cs`	Shared setup for the #10082 Nearby fixtures: stops the grid Driver Apps, opens the Nearby menu, finds this node's Driver App
`Pages/Driver/`	Driver App page objects (`DriverOnboardingPage`, `DriverHomePage`, `DriverReceiptPage`). `DriverHomePage` also covers the current load, its QR and toasts (and the POC's Receive via Bluetooth / truck code, used only while that button exists)
`Flows/`	Steps shared by tests: `CollectionLoadTicketFlow` (Flex login → submitted collection ticket), `LoadDisposalFlow` (Flex disposal: import the Driver's load QR → submitted disposal ticket and its QR), `DriverAppFlow` (Driver App onto Home, restart)
`Tests/QrBaselineTestBase.cs`	Shared by the QR-B and QR-D baseline fixtures: node pair, submitted collection ticket with its decoded QR, damaged payload variants
`Utilities/NearbyDevicesPermission.cs`	Resets, answers and checks Android's *Nearby devices* (Bluetooth) permission
`Utilities/BaselineRecorder.cs`	Saves a run's measurements as JSON under `Baselines/`
`Utilities/NearbyR1Report.cs`, `RequirementsAttribute.cs`, `ReportIndex.cs`	Nearby R1 requirements report (from each test's `[Requirements(...)]`), and `Reports/index.html` linking all reports (see Failure evidence and reports)
`Drivers/DriverFactory.cs`	Starts the browsers (Web) or the Appium sessions (Mobile); forwards backend ports to the emulators
`Utilities/FakeCameraFeed.cs`	Web: feeds an image (QR code) to the browser's fake camera
`Utilities/QrScanFeed.cs`	Hands a QR code to an app's scanner (Web: fake camera; Mobile: see How Mobile mode handles the camera)
`Utilities/AdbHelper.cs`	`adb` helpers: port forwarding and adding photos to the gallery
`Setup/Start-MobileGrid.ps1`	Starts the six node emulators on their fixed ports, then checks each one's Android version and installed APK
`Setup/Build-DebugApk.ps1`	Builds and installs the Flex / Driver debug APKs against the local backend; skips the build when the app's sources didn't change
`Setup/Clear-EmulatorCache.ps1`	Frees disk space: Quick Boot snapshots and emulator temp files, optionally a factory reset (see Freeing disk space)
`Setup/Remove-UnusedEmulatorFiles.ps1`	Frees disk space: broken AVDs and system images no AVD uses

Path	Contents
`Setup/Run-Suite.ps1`	A whole run in one command: Aspire, Appium, grid, APKs, tests, results (see One-command run)
`Setup/mobile.runsettings`	`dotnet test` settings used by `Run-Suite.ps1` (NUnit warnings reported as Passed)
`Setup/azure-pipelines.yml`	Azure DevOps pipeline for a self-hosted agent, local Aspire backend (see Running in Azure DevOps)
`Setup/azure-pipelines-qa.yml`	Azure DevOps pipeline that tests the QA builds after the Flex / Driver App QA pipelines finish
`Config/appsettings.json`	Run mode, URLs, Appium and emulator settings
`TestData/`	Test images (e.g. `DebrisPile.png`)

Setting up a new machine

Nothing has to be copied from an existing machine: the emulators are rebuilt from Google's system images, and the apps, logins and permissions are recreated by the first runs. In this order:

1. **Windows features:** Hyper-V or Windows Hypervisor Platform (Android emulators) and WSL2 (Docker Desktop). Use a machine with 32 GB of RAM for all three nodes.
2. **Tools:** Docker Desktop, .NET 10 SDK, JDK 21, Node.js 20+ with pnpm, VS Code (or Visual Studio), Android Studio with the Android SDK, and Appium with the UiAutomator2 driver. See Tools for the details and versions.
3. **Emulators:** download the six system images and create the six AVDs with **exactly** the names in Emulators. About 22 GB of downloads, plus 10–15 GB per AVD as they're used.
4. **Repos:** clone this repo, the RecoveryTrac Aspire solution and the RecoveryTrac UI repo (Flex and the Driver App). The scripts' defaults are this machine's paths (`C:\Users\adrian.rojas\tetratec\Aspire`, with the UI repo at `RecoveryTrac.UI\apps` inside it). Use the same folders under your user, and pass `-AspirePath` / `-UiPath` to `Run-Suite.ps1` (and `-UiPath` to `Build-DebugApk.ps1`), or clone to exactly those paths.
5. **Per-machine setup:** trust the ASP.NET dev certificate and copy it into the UI repo (Dev certificate), and set the test accounts in user-secrets (Prerequisites). The local database needs the test users (`testuser1`–`4`) set up as described there.
6. **First run:** `.\Setup\Run-Suite.ps1 -RebuildApks`. It starts Docker, Aspire and Appium, starts the grid, builds both debug APKs, installs them on every emulator and runs the BLE tests. The first build takes the longest; later runs only rebuild an app whose sources changed.

Prerequisites (both modes)

- **.NET 10 SDK**
- **The RecoveryTrac Aspire solution running locally** (`RecoveryTrac.AppHost`). It starts the backend services, Flex on `localhost:8100` and the Driver App on `localhost:8200`. Check that both pages open in a browser before running the tests.
- **An ASP.NET Core HTTPS dev certificate** that's trusted (`dotnet dev-certs https --trust`). The local services use it.
- **Test user credentials** in user-secrets (run these in the test project folder):

```
dotnet user-secrets set TestSettings:LoginEmail "<flex test user email>"
dotnet user-secrets set TestSettings:LoginPassword "<flex test user password>"

# Node 2's own Flex account (e.g. Flex.testuser2@tetratech.com)
dotnet user-secrets set TestSettings:Node2:LoginEmail "<second flex test user email>"
dotnet user-secrets set TestSettings:Node2:LoginPassword "<second flex test user password>"

# Node 3's own Flex account
dotnet user-secrets set TestSettings:Node3:LoginEmail "<third flex test user email>"
dotnet user-secrets set TestSettings:Node3:LoginPassword "<third flex test user password>"
```

Environment variables override these, with `__` for `:` (e.g. `TestSettings__Node2__LoginEmail`). The pipeline passes its secrets that way.

A node uses `TestSettings:<node>:LoginEmail` / `LoginPassword` when that email is set. Node 1 falls back to the default account; Node 2 and Node 3 must have their own. Each account needs the same setup: a project and task assignment, and access to the *Load - Collection* and *Load - Disposal* forms.

Configuration (`Config/appsettings.json`)

Setting	Purpose
`TestSettings:RunMode`	`"Web"` or `"Mobile"`
`TestSettings:WebBaseUrl` / `DriverWebBaseUrl`	Flex and Driver App URLs for Web mode (`http://localhost:8100` / `8200`)
`TestSettings:AspireServicePorts`	Local service ports forwarded to the emulators in Mobile mode (see Backend ports)
`Appium:ServerUrl`	Appium server (`http://127.0.0.1:4723`)
`MobileConfigurations:Grid_Node1_Flex`	Node 1 Flex emulator: `Udid` `emulator-5554`, Android 14
`MobileConfigurations:Grid_Node1_Driver`	Node 1 Driver emulator: `Udid` `emulator-5556`, Android 12
`MobileConfigurations:Grid_Node2_Flex`	Node 2 Flex emulator: `Udid` `emulator-5558`, Android 15
`MobileConfigurations:Grid_Node2_Driver`	Node 2 Driver emulator: `Udid` `emulator-5560`, Android 13
`MobileConfigurations:Grid_Node3_Flex`	Node 3 Flex emulator: `Udid` `emulator-5562`, Android 17
`MobileConfigurations:Grid_Node3_Driver`	Node 3 Driver emulator: `Udid` `emulator-5564`, Android 16

Every `Grid_Node*` entry has its own `SystemPort` (820x), `mjpegServerPort` (830x) and `ChromedriverPort` (840x), so parallel sessions on the one Appium server don't share a port.

Running in Web mode

Requirements

- Google Chrome and Microsoft Edge installed. Selenium Manager downloads the matching drivers automatically.
- Aspire running (Flex on `8100`, Driver App on `8200`).

Run

1. Set `"RunMode": "Web"` in `Config/appsettings.json`.
2. Run the test. Web mode doesn't use the node emulators, so run one case. Running both opens two Chrome and two Edge windows at once:

   ```
   dotnet test --filter "Name~E2E_Node1" --logger "console;verbosity=detailed"
   ```

   Or run it from the IDE's Test Explorer.

Chrome (Flex) opens first. Edge (Driver App) opens once the collection QR code exists. A run takes about **2–3 minutes**.

How Web mode handles the camera: both browsers start with a fake camera that plays a file. Before each scan, the test writes the QR code to scan into that file. Photos are attached directly to the page's file input, and the location comes from a mocked browser GPS position.

Running in Mobile mode

One-time setup

1. Tools

Tool	Notes
Docker Desktop (WSL2)	Aspire runs the backend's databases in containers. `Run-Suite.ps1` starts Docker Desktop when it isn't running
.NET 10 SDK	Tests and the Aspire solution
Android Studio + Android SDK	`ANDROID_HOME` must point to the SDK (e.g. `%LOCALAPPDATA%\Android\Sdk`). In SDK Manager → SDK Tools, keep **Android Emulator** and **Android SDK Platform-Tools** installed; add **Android SDK Command-line Tools (latest)** to create the emulators from the command line
Appium 2.x + UiAutomator2 driver	`npm i -g appium` and `appium driver install uiautomator2` (tested with Appium 2.11 and UiAutomator2 2.45)
JDK 21	For the Android Gradle builds (e.g. Eclipse Adoptium 21; `Build-DebugApk.ps1` expects it under `C:\Program Files\Eclipse Adoptium\jdk-21…`, or pass `-JavaHome`). Android Studio's bundled JDK (Java 25) is **too new** and fails with "*Unsupported class file major version 69*"
Node.js 20+ + pnpm	To build the apps' web bundles (tested with Node 22 and pnpm 12)

2. Emulators

Node	Role	Serial	Android	AVD	System image (SDK package)	Device
1	Flex	`emulator-5554`	14 (API 34)	`Pixel_8`	`system-images;android-34;google_apis_playstore;x86_64`	Pixel 8
1	Driver App	`emulator-5556`	12 (API 31)	`Driver_A12_Play`	`system-images;android-31;google_apis_playstore;x86_64`	Pixel 6
2	Flex	`emulator-5558`	15 (API 35)	`Pixel_9`	`system-images;android-35;google_apis_playstore;x86_64`	Pixel 9

Node	Role	Serial	Android	AVD	System image (SDK package)	Device
2	Driver App	`emulator-5560`	13 (API 33)	`Pixel_7_Android_13`	`system-images;android-33;google_apis_playstore;x86_64`	Pixel 7
3	Flex	`emulator-5562`	17 (API 37)	`Pixel_10_Pro`	`system-images;android-37.2;google_apis_playstore_ps16k;x86_64`	Pixel 10 Pro
3	Driver App	`emulator-5564`	16 (API 36)	`Pixel_9_A16`	`system-images;android-36.1;google_apis_playstore;x86_64`	Pixel 9

Creating the emulators (new machine). The AVD names must match the table exactly: `Start-MobileGrid.ps1` starts the emulators by name. All six use **Google Play** images (`google_apis_playstore` , x86_64); the Pixel 10 Pro one is the 16 KB page-size variant (`ps16k`). The images take about 22 GB (Pixel_10_Pro's alone is 8 GB).

- **Android Studio:** Device Manager → *Create Virtual Device* → pick the device from the table → on *System Image* pick the Android version with *Google Play* (for Android 17, the *16 KB Page Size* image) and download it → name the AVD exactly as in the table. Under *Advanced Settings* set **Camera** → **Back: VirtualScene**, and keep 10 GB internal storage.
- **Command line** (needs *Android SDK Command-line Tools*; run with `JAVA_HOME` on JDK 17+), one pair per AVD:

  ```
  $tools = "$env:ANDROID_HOME\cmdline-tools\latest\bin"
  & "$tools\sdkmanager.bat" "system-images;android-34;google_apis_playstore;x86_64"
  & "$tools\avdmanager.bat" create avd -n Pixel_8 -d pixel_8 -k "system-images;android-34;google_apis_playstore;x86_64"
  # ...same for the other five rows (devices: pixel_6, pixel_9, pixel_7, pixel_10_pro, pixel_9)
  ```

  Then, in each `%USERPROFILE%\.android\avd\<name>.avd\config.ini` , set `hw.camera.back=virtualscene` , `hw.gpu.mode=host` and `disk.dataPartition.size=10G` .
- **Check:** `.\Setup\Start-MobileGrid.ps1` starts all six and shows each one's Android version. Until the apps are installed it reports them missing; `Run-Suite.ps1 -RebuildApks` installs them.

Nodes 1 and 2 have passed full parallel runs in both Web and Mobile mode. Node 3 hasn't completed a run yet.

- **Start the grid with the script.** It starts each emulator on its own port, so the serial always matches `appsettings.json` . (Started by hand, the serial depends on start order: the first emulator gets 5554.) It stops if a port is taken by the wrong AVD. Once the emulators have booted, it reports each serial's Android version and whether its APK is installed:

  ```
  .\Setup\Start-MobileGrid.ps1               # all 3 nodes, windows visible
  .\Setup\Start-MobileGrid.ps1 -Nodes 1,2    # only some nodes (saves RAM)
  .\Setup\Start-MobileGrid.ps1 -Headless     # no windows
  .\Setup\Start-MobileGrid.ps1 -Stop         # shut the grid down
  .\Setup\Start-MobileGrid.ps1 -MemoryMB 3072 # more RAM per emulator
  .\Setup\Start-MobileGrid.ps1 -Arrange      # lay out the running emulators' windows side by side again
  ```

  The script lays the emulator windows out side by side across your monitors, preferring monitors tall enough for a whole device. Each AVD otherwise opens where it was last, which can be stacked or off the top of a short screen.

Every emulator starts with 2 GB of RAM and 2 CPU cores (`-MemoryMB` , `-Cores`), whatever its AVD says. That keeps all six within a 32 GB laptop. The script warns before starting an emulator when there isn't enough free memory for it.

The AVD-to-port table is at the top of the script; keep it in sync with `appsettings.json` . To check a serial by hand:

```
adb devices
adb -s emulator-5558 emu avd name
adb -s emulator-5558 shell pm list packages | findstr tetratech
```

- **Memory:** four emulators already use most of a 32 GB machine, so six is tight. Close Android Studio and browsers before a parallel run, and start only the nodes you run (`-Nodes`). Frozen emulators or Appium session timeouts usually mean the host is out of RAM.
- **Back camera:** the Collection Start and Load Call photo fields need a live photo, so the Flex emulator must have a working camera. Pixel_8 uses `hw.camera.back=virtualscene` ; a real webcam (`webcam0`) also works.
- **GPU:** `hw.gpu.mode=host` is set on the emulators (`config.ini` in `%USERPROFILE%\.android\avd\<name>.avd`).
- **Android 17 (`Pixel_10_Pro`)** uses a 16 KB page-size system image (`ps16k`). If Flex won't install or crashes on start there, a native library in the APK isn't 16 KB-aligned. That's a problem with the app build, not the tests.

3. Debug APKs that talk to the local backend

The tests need **debug builds** of Flex and the Driver App that call the local Aspire services. Store or release builds won't work. Build and install them with:

```
.\Setup\Build-DebugApk.ps1 -App Flex   -Udid emulator-5554   # builds and installs on Node 1 Flex
.\Setup\Build-DebugApk.ps1 -App Driver -Udid emulator-5556   # builds and installs on Node 1 Driver
```

The script's default serials (Flex `5556` , Driver `5554`) are from the older single-pair layout, so pass `-Udid` . To put the same build on Node 2 and Node 3, install the APK directly:

```
$ui = "C:\Users\adrian.rojas\tetratec\Aspire\RecoveryTrac.UI\apps"
adb -s emulator-5558 install -r "$ui\Flex\android\app\build\outputs\apk\debug\app-debug.apk"
adb -s emulator-5560 install -r "$ui\Driver-App\android\app\build\outputs\apk\debug\app-debug.apk"
adb -s emulator-5562 install -r "$ui\Flex\android\app\build\outputs\apk\debug\app-debug.apk"
adb -s emulator-5564 install -r "$ui\Driver-App\android\app\build\outputs\apk\debug\app-debug.apk"
```

The script expects the RecoveryTrac UI repo at `C:\Users\adrian.rojas\tetratec\Aspire\RecoveryTrac.UI` (override with `-UiPath`) and JDK 21 at the Eclipse Adoptium path (override with `-JavaHome`). When it builds, it always does a **clean** Gradle build, because an incremental build once produced an APK that crashed on start (*ClassNotFoundException: MainActivity*).

Skipping unchanged builds: before building, the script fingerprints what the APK is built from: the app's folder, `libs/` and the repo's root files (committed, modified or untracked), plus the script's own settings. Other apps, docs, Markdown files and the files `cap sync` rewrites don't count. When the fingerprint matches the last build's (`app-debug.apk.fingerprint` next to the APK), it skips the build and installs the existing APK. Add `-Force` to build anyway.

What the script does:

- **Flex:** builds the web bundle with every service URL set to its local Aspire port (`NG_APP_*` variables, the same ones Aspire passes to web Flex), then `cap sync android`, `gradlew clean assembleDebug` and install.
- **Driver App:** its `environment.ts` already points at `https://localhost:7013` and `https://localhost:7107`, so it builds as is.

Dev certificate (one-time, per machine): the debug APKs must trust this machine's ASP.NET dev certificate to call `https://localhost:<port>`. Export it without the private key:

```
dotnet dev-certs https -ep aspnet-dev.crt --format PEM --no-password   # then delete the generated .key file
```

Copy the certificate into the UI repo. **Never commit it**: it's specific to your machine.

App	Certificate file	Trusted by
Flex	`apps/Flex/android/app/src/main/res/raw/aspnet_dev_cert.crt`	`res/xml/network_security_config.xml` (`debug-overrides`); the manifest points to it via `android:networkSecurityConfig`
Driver App	`apps/Driver-App/android/app/src/debug/res/raw/aspnetcore_dev_cert.crt`	The Driver App's existing debug source set (`src/debug/res/xml/network_security_config.xml`)

If the installed app is a release build, uninstall it once before the first debug install (`adb -s <serial> uninstall <package>`), because the signatures differ.

Backend ports

Each emulator reaches the local services as `https://localhost:<port>` through `adb reverse`. **The test sets this up itself** for every port in `TestSettings:AspireServicePorts`, right after each emulator session starts.

Port	Service
52001 / 52002 / 52003 / 52004	Activity / Attachment / Audit / Auth
52007 / 52010 / 52011 / 52012	Form / Lookup / Mobile / Notification
52015 / 52017 / 52019 / 52020 / 52021	Project / Role / TimeRecord / Time / User
7013	Driver service (Driver App)
7107	RecoveryTrac Web API

The local AppHost runs DriverService on 52006, not 7013, so nothing answers on 7013 locally. The Driver App's `environment.ts` still calls 7013, so its DriverService calls fail locally; the BLE handoff only uses the Web API (7107). `Run-Suite.ps1` doesn't wait for 7013 (`-OptionalPorts`).

If Aspire ever assigns different ports, update this list and the port table in `Setup/Build-DebugApk.ps1`. The real values are in the web bundle on `localhost:8100` (search it for `localhost:52`). If a port is missing, the app shows "*Unable to reach server*" at login.

Before each Mobile run

1. **Aspire** is running.
2. **The node emulators** are running on the serials in `appsettings.json`, with the debug APKs installed (Flex on 5554/5558/5562, Driver on 5556/5560/5564). `.\Setup\Start-MobileGrid.ps1` shows `OK` for each one when this is true.

3. **Appium** is running, in its own terminal:

   ```
   appium --address 127.0.0.1 --port 4723 --allow-insecure chromedriver_autodownload
   ```

   The `chromedriver_autodownload` flag lets Appium download the chromedriver that matches each emulator's WebView. Newer Appium versions require the driver prefix: `--allow-insecure uiautomator2:chromedriver_autodownload`.

4. **Run mode:** `"RunMode": "Mobile"` in `Config/appsettings.json`.

Run

```
# All three nodes in parallel
dotnet test --filter "Name~LoadDisposalCollectionFlow" --logger "console;verbosity=detailed"

# Some nodes only
dotnet test --filter "Name~E2E_Node1|Name~E2E_Node2" --logger "console;verbosity=detailed"
dotnet test --filter "Name~E2E_Node3" --logger "console;verbosity=detailed"
```

The cases run at the same time: `[Parallelizable(ParallelScope.Children)]`, up to `LevelOfParallelism(3)` in `Properties/AssemblyInfo.cs`, which is exactly three nodes. Raise it if you add a fourth node. Each case uses the node pair it names, not the shared node pool. A run takes about **4–5 minutes** per node, and the parallel run takes about as long as the slower node. Leave the emulators alone while it runs.

QR baselines on Node 2 and Node 3 in parallel (`.\Setup\Start-MobileGrid.ps1 -Nodes 2,3`):

```
dotnet test --filter "Category=QR-Baseline" --logger "console;verbosity=normal"           # QR-B and
QR-D
dotnet test --filter "Category=QR-Baseline-Disposal" --logger "console;verbosity=normal"   # QR-D only
dotnet test --filter "Name~QR_D04_D05"                                                     # one
scenario
```

The QR-B tests take about **12 minutes** per node. Each QR-D test submits a collection and a disposal ticket: QR-D01…D03 took about **3.5 minutes** on Node 2, and the other QR-D tests haven't been timed yet (QR-D09 submits two collection tickets, so it takes longer).

BLE handoff (TC-12 / TC-13), Node 2 then Node 3 (one after the other). Only those two node pairs need to be running (`.\Setup\Start-MobileGrid.ps1 -Nodes 2,3`):

```
dotnet test --filter "Category=BLE" --logger "console;verbosity=normal"
dotnet test --filter "FullyQualifiedName~BLE_TC12&FullyQualifiedName~Node3"   # one node only
```

A run takes about **4–5 minutes** per node. On a pass, the console shows the Driver App card Flex picked, the send status and the timing, for example:

```
📡 Flex lists this node's Driver App as 'GQZS' (Driver App / ~0 ft away / Identity Code: GQZS) after
0.4 s.
📡 Flex: 'Sent to GQZS' after 2.9 s.
🚚 Driver app: current load 65Z9-LC-1002 after 3.2 s (toast: 'Load 65Z9-LC-1002 received.').
```

#10082 R1-C1 discoverability (NDT-1) needs only Node 2 (`.\Setup\Start-MobileGrid.ps1 -Nodes 2`). Run it on its own, about **2–3 minutes per test**:

```
dotnet test --filter "Category=BLE-R1-C1" --logger "console;verbosity=normal"
```

#10082 R1-C2 transfer core also needs only Node 2, about **2–3 minutes per test**. The NDT-5 test only runs with `TestSettings:RunMode=Web` (Flex Web); in a Mobile run it is ignored, and the device tests are ignored in a Web run. So the category takes two passes, and neither needs `appsettings.json` edited:

```
.\Setup\Run-Suite.ps1 -Filter "Category=BLE-R1-C2" -Nodes 2              # the device tests (NDT-5
ignored)
.\Setup\Run-Suite.ps1 -Filter "Name~NDT5" -RunMode Web                   # NDT-5 in Chrome / Edge
```

Mobile and Web in one run: the tests read environment variables last, so `TestSettings__RunMode=Web` (or `Mobile`) for one run overrides `Config/appsettings.json` . `Run-Suite.ps1 -RunMode` sets it for you. Keep `"RunMode": "Mobile"` in `appsettings.json` , so Test Explorer and plain `dotnet test` stay on the emulators, and set the variable only for the browser tests:

```
$env:TestSettings__RunMode = "Web"; dotnet test --filter "Name~NDT5"; Remove-Item
Env:TestSettings__RunMode
```

A run whose load took longer than 5 s ends with a **warning** outcome. The `dotnet test` summary doesn't count that as passed (Test Explorer shows it as Skipped), but all of its checks held. The recorded JSON shows its numbers. `Run-Suite.ps1` passes `Setup/mobile.runsettings` , which reports warnings as Passed; add `--settings Setup/mobile.runsettings` to a plain `dotnet test` for the same.

All nodes pick the **first truck**. If the backend ever allows only one active load per truck, every node but the first will fail at the Driver scan step.

One-command run (`Setup/Run-Suite.ps1`)

Does everything in Before each Mobile run, then runs the tests:

1. **Aspire:** uses it when its service ports answer, and waits when only some do (an AppHost still starting). Otherwise it starts `RecoveryTrac.AppHost` and waits for the ports. If Docker isn't running, it starts Docker Desktop first and waits for it (up to 3 minutes).
2. **Appium:** uses the one on port 4723, or starts it with the right `--allow-insecure` form for its version.
3. **Grid:** `Start-MobileGrid.ps1 -Nodes ...` . When step 4 installs the apps, an emulator still missing one is reported (*not installed yet*) but doesn't stop the run.
4. **Apps:** with `-RebuildApks` , builds both debug APKs (`Build-DebugApk.ps1` , skipping an app whose sources didn't change since its last build; `-ForceRebuild` builds anyway) and installs them on every node. With `-FlexApk` / `-DriverApk` , installs those. Otherwise keeps what's installed.
5. **Bluetooth:** on each emulator of the run, closes crash dialogs and restarts Bluetooth when it is off, when Android's Bluetooth service crashed, or when the shared Bluetooth simulator (`netsimd`) restarted after the emulator booted. That restart makes every running emulator's Bluetooth abort ("*Bluetooth keeps stopping*", `Hardware Error Event with code 0x42`). If Bluetooth doesn't come back, cold-boot the grid: `.\Setup\Start-MobileGrid.ps1 -Stop` , then `.\Setup\Start-MobileGrid.ps1 -Nodes 2,3 -ColdBoot` .
6. **Tests:** `dotnet test` with `-Filter` and `Setup/mobile.runsettings` , with `TestSettings:RunMode` set to `-RunMode` (default `Mobile`). With `-RunMode Web` , steps 2–5 are skipped: the tests open Flex and the Driver App in Chrome and Edge against the local Aspire (`Target Local` only).

7. **Results** go to one folder (default `TestResults/run`): `mobile-tests.trx` (or `web-tests.trx` for `-RunMode Web`, so both passes keep their results), the HTML reports, baselines, failure screenshots, and the Aspire / Appium logs when it started them.

```
.\Setup\Run-Suite.ps1                                    # Category=BLE on nodes 2,3, apps as installed
.\Setup\Run-Suite.ps1 -RebuildApks                       # debug APKs from the UI repo first (only the
apps that changed)
.\Setup\Run-Suite.ps1 -RebuildApks -ForceRebuild         # build both even when nothing changed
.\Setup\Run-Suite.ps1 -Filter "Category=BLE-R1-C1" -Nodes 2
.\Setup\Run-Suite.ps1 -Filter "Category=BLE-R1-C2" -Nodes 2
.\Setup\Run-Suite.ps1 -Filter "Name~NDT5" -RunMode Web       # browser tests: Aspire only, no Appium /
emulators
.\Setup\Run-Suite.ps1 -Target QA -FlexApk <flex.apk> -DriverApk <driver.apk>   # QA services, no Aspire
.\Setup\Run-Suite.ps1 -Teardown                          # stop the grid, and Appium / Aspire if it
started them
.\Setup\Run-Suite.ps1 -Stop                              # only that teardown
```

The exit code is the `dotnet test` one (non-zero when a test failed). `-AspirePath` (default `C:\Users\adrian.rojas\tetratec\Aspire`) and `-UiPath` (default its `RecoveryTrac.UI\apps`) point at other checkouts. `-Target QA` needs APKs built against the QA services; the BLE code isn't in QA yet, so use `Local` for the BLE tests.

Freeing disk space

The emulators fill the disk over time (Quick Boot snapshots of 2–4 GB per AVD, user data up to 10 GB per AVD, unused system images). Two scripts clean them up. Both refuse to run while an emulator is open, so stop the grid first, and both accept `-WhatIf` to list what would go and how big it is.

```
.\Setup\Start-MobileGrid.ps1 -Stop
.\Setup\Clear-EmulatorCache.ps1 -WhatIf                  # preview
.\Setup\Clear-EmulatorCache.ps1                          # Quick Boot snapshots, lock files, emulator
temp / crash files
.\Setup\Clear-EmulatorCache.ps1 -WipeData                # also factory-reset the AVDs (biggest saving)
.\Setup\Clear-EmulatorCache.ps1 -Avd Pixel_8,Pixel_9     # only these AVDs
.\Setup\Remove-UnusedEmulatorFiles.ps1 -Confirm:$false   # broken AVDs (no config.ini) + system images
no AVD uses
.\Setup\Remove-UnusedEmulatorFiles.ps1 -RemoveAvd Old_Phone   # also delete a working AVD you name
```

- **Clear-EmulatorCache:** everything it deletes is rebuilt by the emulator. The next start is a cold boot (about a minute longer). With `-WipeData` the apps, their data, granted permissions and the Google sign-in are gone: reinstall the APKs (`Run-Suite.ps1 -RebuildApks` handles emulators without the apps) before the next run.
- **Remove-UnusedEmulatorFiles:** keeps every system image a remaining AVD is built on. A deleted image can be downloaded again from Android Studio's SDK Manager.
- **Outside the emulators:** the UI repo's Angular build cache (`apps/*/.angular`, often 10+ GB) and `%USERPROFILE%\.gradle\caches` are safe to delete; the next build just takes longer.

How Mobile mode handles the camera

Step	Web	Mobile
Photos	Attached to the file input	Taken with the emulator camera (both fields block gallery uploads)
QR scans (Driver × 2, Flex × 1)	The browser's fake camera shows the QR	**Scanner hand-off** (below)
Location	Mocked browser GPS	GPS fix sent through the emulator console (see Location)

Scanner hand-off (`Utilities/QrScanFeed.cs`):

- **Why:** the emulator cameras can't show a chosen image on Android 12. The virtual scene camera only shows a black screen on Android 12 images, and a host virtual webcam such as OBS isn't visible to the emulator on Windows.
- **How:** the test reads the QR image with ZXing.Net. Inside the app's WebView it then answers the ML Kit `BarcodeScanner` plugin: `startScan` reports the QR text once through the normal `barcodesScanned` event, instead of opening the camera.
- **Coverage:** everything after the scan runs exactly as on a real device (validation, the load import, navigation). Only the camera reading the code is bypassed.
- **QR images are read from the page, not screenshotted.** The Driver App draws its load QR as a PNG data URL, and the test takes those exact bytes. On the Android 12 emulator, an element screenshot came out cropped by a few pixels: it clipped a corner square, and the code couldn't be decoded.

Location

`CollectionLoadTicketFormPage.CaptureLocation`:

- Appium's `mobile: setGeolocation` mock does **not** reach the app on these emulators. They keep reporting the default fix (Google HQ, 37.4220, -122.0840).
- So the test sends the fix through the emulator console instead: `adb -s <serial> emu geo fix <lon> <lat>`, right before tapping the GPS button and every 2 seconds while waiting. A console fix arrives as a real GPS reading.
- The location field only accepts a fix within 15 ft. The emulator's console fix sometimes reports 16 ft; the app then keeps the old location and offers **Use this location**. The test taps it rather than falling back to the map. The map opens on the centre of the US (Kansas), where the address lookup finds nothing.
- If Street Name stays empty (the address lookup failed), the test sends the fix again and taps GPS once more. If that also fails, it stops with a clear message instead of timing out.
- The address lookup for the mock point sometimes returns just "*Road*" as the street name. That passes, because the test compares the ticket with whatever the form filled in.

Other emulator-specific handling

- **Login:** on an emulator, the LOG IN tap sometimes only focuses the button. The test checks that the form went away and taps once more if needed. If it's still on the form, it fails and lists the error messages the app shows.
- **Driver App launch after a cold boot:** on the first session after an emulator boots, Appium's `am start` can hang and leave the launcher in front. `DriverAppFlow.StartOnHome` then opens the Driver App itself ("*Driver app not in front after the session started*").
- **Driver App toasts:** a toast such as "*Load … received.*" closes after a couple of seconds, and with two node pairs running it could be gone before the test read it. The BLE test therefore records toasts as the app

shows them (`DriverHomePage.StartRecordingToasts`), starting just before it sends.

- **Driver App tutorial (Android 13+):** SKIP is drawn under the status bar and is missing from the native screen layout. The test recognises the tutorial by its slides and taps SKIP inside the app's WebView.
- **Assignment with a filled-in Position:** some accounts show *Position* instead of *Labor Category*. The test re-selects the Position so the assignment passes validation and the modal closes.

Running in Azure DevOps

`Setup/azure-pipelines.yml` runs `Run-Suite.ps1` on one **self-hosted** Windows agent. Microsoft-hosted agents can't run it: they have no nested virtualization or room for four emulators, and the BLE tests need every emulator on one host (they share one simulated Bluetooth).

Agent (pool `MobileGrid`, to be created):

Item	Requirement
Machine	Windows 11 / Server 2022 with nested virtualization (Azure Dv5/Ev5 class: 16 vCPU, 64 GB RAM, 256 GB+ SSD)
Virtualization	Hyper-V, Windows Hypervisor Platform (emulators), WSL2 (Docker)
Tools	Docker, .NET 10 SDK, JDK 21, Node + pnpm, Appium 2+ with UiAutomator2, Android SDK with the grid's AVDs
Agent account	The user that owns the AVDs, running interactively with auto-logon (a service account can't see the AVDs or show a session)
Aspire	The AppHost's settings / user secrets (e.g. `Parameters:SqlServerPassword`), the dev certificate trusted and copied into the UI repo as in Dev certificate
Test data	The test accounts (`testuser1`–`4`) in the agent's local database, with the same setup as in Prerequisites

Pipeline setup:

- A variable group `MobileAutomation-Secrets` with the secrets `TestSettings-LoginEmail`, `TestSettings-LoginPassword`, `TestSettings-Node2-LoginEmail`, ... (mapped to `TestSettings__...` in the YAML).
- A GitHub service connection (`GitHub-TetraTec` in the YAML) to check out `recoverytrac-ble-poc`, as long as the BLE code is only there. Parameter `uiSource: aspireSubmodule` builds the UI submodule the Aspire repo pins instead.
- The job's build service account needs read access to the Aspire, Services, DataHub, Documentation and RecoveryTrac.UI repos. The YAML rewrites the submodules' SSH URLs to HTTPS, because the checkout task can't use SSH.

Mobile and browser tests in one pipeline run: parameter `filter` runs on the emulators (`Run-Suite.ps1 -RunMode Mobile`). Parameter `webFilter` (default `none`) adds a second pass in Chrome and Edge (`-RunMode Web`) against the same Aspire, e.g. `filter: Category=BLE-R1-C2` with `webFilter: Name~NDT5`. The mobile step then leaves Aspire running and the browser step tears everything down; each pass publishes its own test run (`mobile-tests.trx`, `web-tests.trx`). The browser pass needs Chrome and Edge on the agent.

Against QA (`Setup/azure-pipelines-qa.yml`)

Once Flex and the Driver App are deployed to QA, this pipeline tests the QA builds instead of a local backend. It runs when `RecoveryTrac.UI - Flex - QA` or `RecoveryTrac.UI - Driver-App - QA` completes, downloads the APK each one published (the latest successful one for the pipeline that didn't trigger the run), and runs `Run-Suite.ps1 -Target QA`. That installs both APKs on the grid and runs the tests, with **no Aspire, Docker or dev certificate**: the apps call the QA services.

Same agent pool, but it needs only the emulator side of the table above: drop Docker, WSL2 and the Aspire items. Before the first run:

- Both QA pipelines publish their APK as a pipeline artifact. Set `flexArtifact` / `driverArtifact` in the YAML to those artifact names.
- The APKs are built for QA, include the BLE handoff code, and keep the package names the tests expect (`com.tetratech.recoverytracflex`, `com.tetratech.recoverytracdriver`).
- A variable group `MobileAutomation-QA-Secrets` holds the **QA** test accounts, with the same names as `MobileAutomation-Secrets`. The accounts need the same setup in QA as locally.

The run title in the **Tests** tab names the Flex and Driver App QA builds it tested.

Schedule and results

The local-Aspire pipeline runs nightly on weekdays and by hand (parameters: filter, nodes, UI source, rebuild APKs). Results appear in the run's **Tests** tab; the HTML report, baselines, screenshots and logs are in the `mobile-results-<attempt>` artifact. A failed test fails the run.

Failure evidence and reports

- **Failure files:** when a test fails, the screen, page text and page source of each app are saved to `bin/Debug/net10.0/` as `Failure_<test>_<timestamp>_Flex.*` and `..._DriverApp.*`. `<test>` ends in the node name (e.g. `..._E2E_Node2_A15_A13`). They're also attached to the test result. The `.html` file is the most useful one for fixing locators.
- **Passwords are masked:** before the files are saved, the text of every password field, and the login password wherever it appears, is replaced with `********`. Failure files saved before this change may contain the password in plain text. Delete them and don't share them.
- **Reports:** always written to **`TestResults/Reports/`** at the project root, whether the tests ran from Test Explorer (Debug) or `Run-Suite.ps1` (Release), so every run adds to the same reports. `Run-Suite.ps1` also copies them into its results folder (e.g. the pipeline artifact). Start from **`TestResults/Reports/index.html`**, which links the reports with when each was last written:

Report	Covers	Written
`NearbyR1Report_latest.html`	Feature #10082 R1: every NDT requirement with its status (**Met**, **Not met**, **Pending**, **Not checked**, **Not run**), the tests that check it and the finding that named it; test results, baselines, devices, failure screenshots	After a run with tests marked `[Requirements(...)]`
`QrBaselineReport_latest.html`	BLE POC Part A: QR-A01, QR-B01…B11, the disposal baselines QR-D01…D10 and TC-12 / TC-13, their requirement checks, baselines, devices	After a run with any of those tests

  The Nearby and QR reports each list only their own tests, and keep the latest result of each test across runs (`*_results.json` next to them), so running one category updates only its rows. To start from scratch, delete the `Reports` folder.

- **A requirement's status in the Nearby report** comes from its tests' results: *Not met* when a result names it (`10082-NDT-2-P14 / N04 not met: …`), *Pending* when that happens in a `*-Pending` category, *Not checked* when a warning says so or the test failed before its checks (e.g. a device not responding), *Met* when the test ran and didn't report it. A new Nearby test needs `[Requirements("NDT-x-yyy", ...)]` with the IDs it checks; a control test gets `[Requirements]` with none.

Troubleshooting

Symptom	Cause / fix
`Activity name '.com.tetratech.recoverytracflex.MainActivity' ... doesn't exist`	The emulator serials don't match the installed apps. Check `adb devices` and the `Udid` values in `appsettings.json`
`No connection could be made ... 127.0.0.1:4723`	Appium isn't running
`UiAutomator2 Server cannot start because the local port #8201 is busy`	A stopped or aborted run left its Appium session behind. Restart Appium and wait about a minute
Login shows "*Unable to reach server*"	Aspire isn't running, or a service port is missing from `AspireServicePorts`
Driver App crashes on start (`ClassNotFoundException`)	Rebuild with `Setup/Build-DebugApk.ps1 -App Driver` (clean build)
Gradle: "*Unsupported class file major version 69*"	Use JDK 21 (`-JavaHome`), not Android Studio's bundled JDK
Build: "*file is being used by another process*"	A debug session is holding the test DLL. Stop it (Shift+F5)
Driver App still has a load from an earlier run	The test cancels it automatically (it waits out the CONFIRM countdown)
`LoginPassword setting is missing ... TestSettings:Node2:LoginPassword`	Node 2's email is set but its password isn't. Set it with `dotnet user-secrets set TestSettings:Node2:LoginPassword <value>`
Node 2 fails at login or finds no project	The Node 2 account isn't set up like the Node 1 account (project/task assignment, form access)
Emulators freeze or sessions time out only in parallel runs	The host is out of RAM. Close other apps, start fewer nodes (`Start-MobileGrid.ps1 -Nodes 1,2`), and filter the run to those nodes
An emulator window is missing from the screen	It opened off-screen or behind another window. Run `.\Setup\Start-MobileGrid.ps1 -Arrange`
`Start-MobileGrid.ps1` : *... is not running at all: it failed to start*	Usually not enough free RAM for another emulator. Run fewer nodes (`-Nodes 1,2`), or close Android Studio, WSL and browsers
`Node3 has no Flex account of its own`	Set `TestSettings:Node3:LoginEmail` and `LoginPassword` in user-secrets (same for Node2)
`Start-MobileGrid.ps1` : *emulator-55xx is running 'X', not 'Y'*	The emulators were started on the wrong ports. Run `.\Setup\Start-MobileGrid.ps1 -Stop` , then start the grid again
`Login as '...' did not leave the Log In screen`	Wrong password for that node's account, or the backend can't be reached. The message lists the error text the app showed
emulator-55xx cannot resolve host names (DNS is down)	The emulator kept the host's DNS servers from boot time, and the host's network (VPN, Wi-Fi) changed since. Run `.\Setup\Start-MobileGrid.ps1 -Stop` and start the grid again; the script pins `-dns-server 8.8.8.8,1.1.1.1`

Symptom	Cause / fix
Street Name was not filled in … (reverse geocoding failed twice)	The emulator can't reach the address lookup (ArcGIS), or the GPS fix never arrived. Check the emulator's network, or run `adb -s <serial> emu geo fix -81.69 28.16` while the form is waiting
`No QR code could be read from the staged image`	The QR image couldn't be decoded. Check the `QrCodes/*.png` attachment for a cropped or blank image
Driver App stuck on the tutorial (Android 13+)	The WebView SKIP tap failed. Look for the SKIP button in the Driver failure `.html`
`Grid_Node2_Driver (emulator-5560) is not responding to adb`	That emulator isn't running. Check with `adb devices` and `adb -s <serial> emu avd name`. Emulators started by hand take the first free port, so an AVD can end up on another node's port. Fix it with `.\Setup\Start-MobileGrid.ps1 -Stop`, then start the grid with the script
`No configured Flex account matches 'testuser4'`	A BLE fixture names an account that isn't in user-secrets. Set it under any `TestSettings:<name>:LoginEmail` / `LoginPassword`
BLE: *Precondition not met: Flex did not list this node's Driver App within 25 s*	The Driver App isn't advertising: its Bluetooth is off, or it lacks Nearby devices access (`BLUETOOTH_ADVERTISE` / `BLUETOOTH_CONNECT`; the tests grant it, opening the app by hand doesn't). Grant it with `adb -s <udid> shell pm grant com.tetratech.recoverytracdriver android.permission.BLUETOOTH_ADVERTISE` (same for `BLUETOOTH_CONNECT`)
BLE: *Flex could not send ticket … to the Driver App*	Flex shows the failure reason instead of *Sent to …*. Check that the Driver emulator is responsive and run again
Flex: "*Unable to create a new form document.*" (log: `reading 'deviceID'`)	Flex was reinstalled but kept its old login, so the post-login sync that creates its device ID never ran. Clear Flex's data (`adb -s <udid> shell pm clear com.tetratech.recoverytracflex`) so the test logs in fresh
Flex shows *No nearby Driver Apps found.* when testing by hand	The Driver App must be open (foreground) and have Nearby devices access, see above
`10082-NDT-1-… not met`	The Driver App wasn't found in that state. If `NDT1_Control_…` also fails, the search itself is broken: check that Flex's Nearby menu opens, that both emulators' Bluetooth is on, and that the Driver App has Nearby devices access
`10082-NDT-2/3/5/6-… not met`	Flex doesn't meet that requirement in the installed build (e.g. NDT-5: Flex Web still shows the Bluetooth icon). A `Precondition not met: Flex did not list this node's Driver App` failure is different: the Driver App isn't advertising (check `NDT1_Control_…` and the Driver emulator's Bluetooth)
`10082-NDT-10-… not met` (category `BLE-R1-C2-Pending`)	Expected until Flex ships Story 4.8: the send view has no *Send Data* title, no spinner states and no Cancel yet
Flex emulator left with Bluetooth off	An NDT-6 test was killed before its `finally` ran. Turn it back on with `adb -s <udid> shell cmd bluetooth_manager enable`
BLE: a fixture shows neither *Passed* nor *Failed*	The run passed every check but took over 5 s, so it ended with a warning. See `Baselines/BLE-TC12_<node>_*.json`